



ARQUITECTURA DE SOFTWARE II

Práctico Clase 2: MongoDB, Go y el Repository Pattern

Desacoplamos la lógica de negocio de la infraestructura de datos mediante interfaces, repositorios reales con MongoDB y mocks para tests.

Paquete DAO: Modelado de Datos para MongoDB

El paquete DAO (Data Access Object) es fundamental para definir cómo nuestra aplicación Go interactúa con MongoDB, estableciendo las estructuras de datos que reflejan nuestros documentos.

Estructuras Go

Define `structs` de Go que mapean directamente a los documentos de MongoDB, permitiendo una interacción tipada.

Seguridad de Tipos

Garantiza que los datos se almacenen y recuperen con el formato y los tipos correctos, minimizando errores en tiempo de ejecución.

Abstracción de Persistencia

Separa la lógica de negocio de los detalles de cómo se guardan o se leen los datos de la base de datos.

AGENDA

¿Qué vamos a aprender hoy?

01

MongoDB con Docker

Levantar una instancia local y ejecutar operaciones básicas

02

Acceso desde Go

Conectar la aplicación con MongoDB programáticamente

03

Interfaces en Go

Definir contratos que abstraen el acceso a datos

04

Repository Pattern

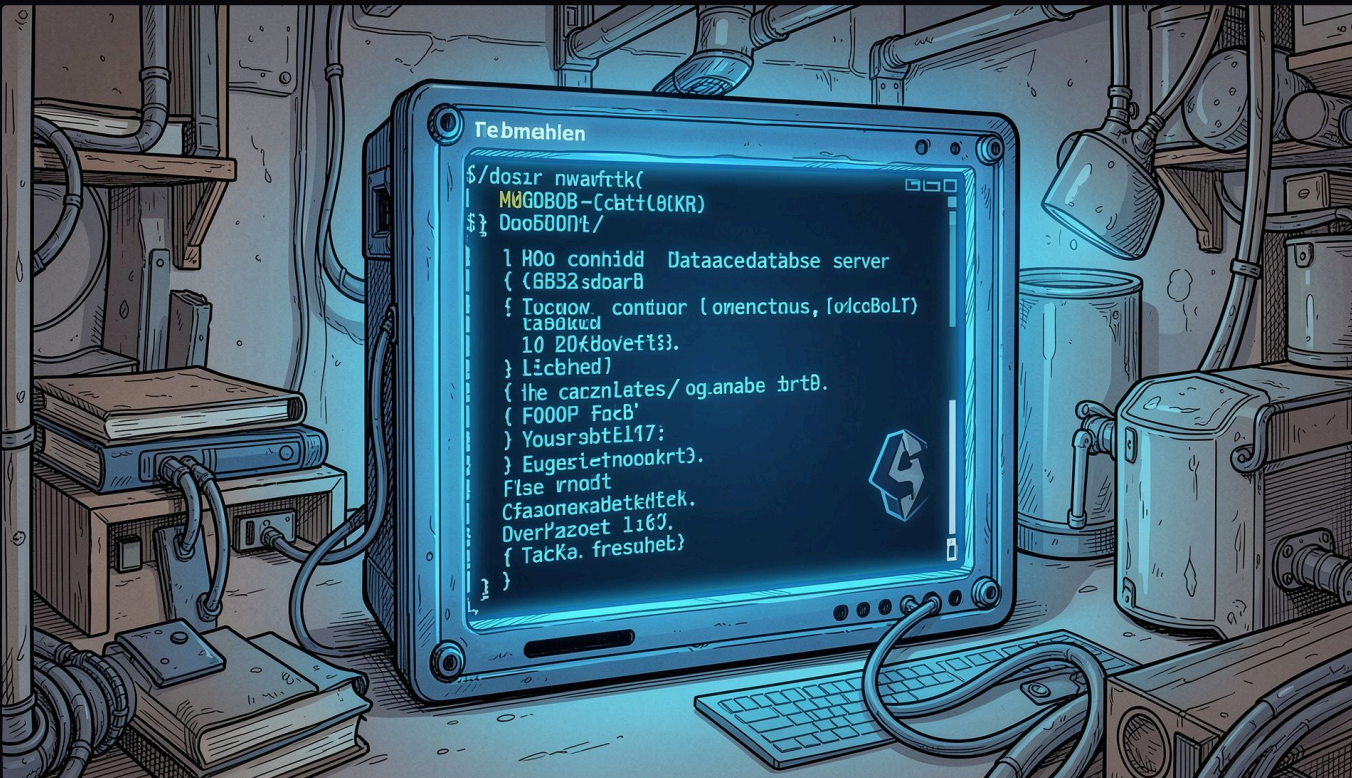
Implementación real + Mock para tests independientes

05

API de Items

Aplicar todo sobre una API real

Levantar MongoDB con Docker



1. Descargar la imagen

```
docker pull mongo:4.4.6
```

2. Ejecutar el contenedor

```
docker run -p 27017:27017 \
  -e
  MONGO_INITDB_ROOT_USERNAME
  =root \
  -e
  MONGO_INITDB_ROOT_PASSWORD
  =root \
  --name my-mongo mongo:4.4.6
```

- ❏ Expone el puerto 27017 y configura las credenciales **root/root**.

Acceder a MongoDB

Para interactuar con la instancia de MongoDB que hemos levantado con Docker, sigue estos pasos:

1. Verificar que el contenedor está ejecutándose

```
docker ps
```

2. Ingresar al contenedor

```
docker exec -ti my-mongo bash
```

3. Iniciar el cliente de MongoDB

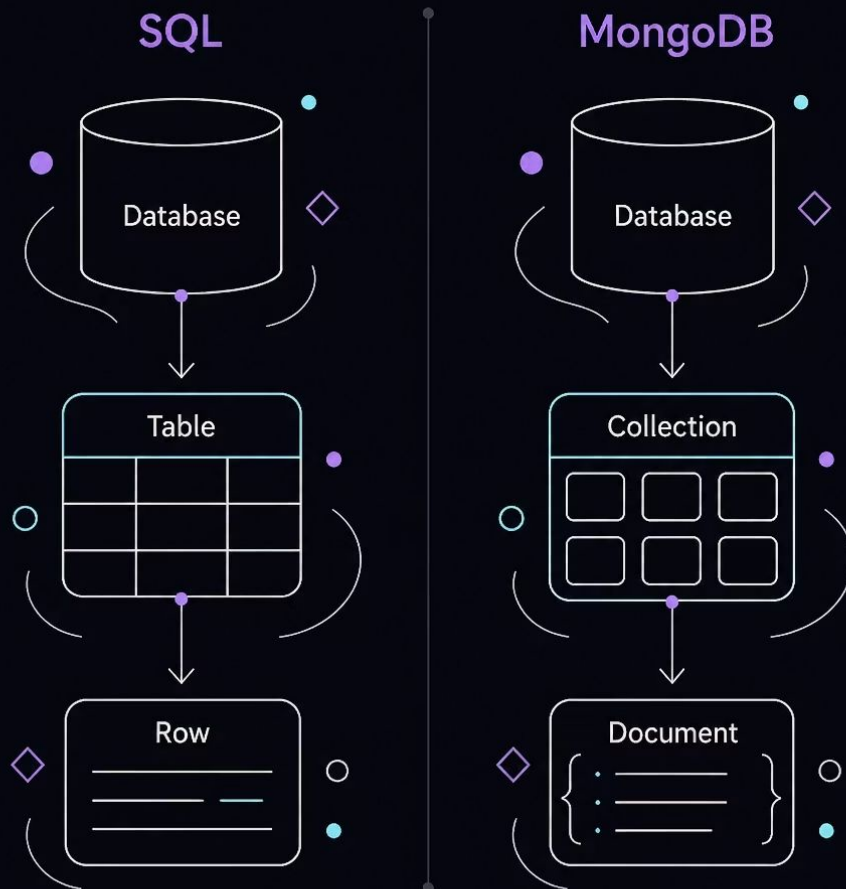
```
mongo --username root --password root --authenticationDatabase admin
```

4. Verificar la conexión

```
show dbs
```

Si podemos listar las bases de datos, nuestra instancia de MongoDB está funcionando correctamente.

Primeros pasos: el modelo de datos



Modelo documental

MongoDB organiza la información en **documentos** dentro de **colecciones**, sin esquemas fijos.

```
use hotels
db.hotels.find().pretty()
db.hotels.find({ name: "Grand Hotel" })
```

Los documentos soportan strings, números, fechas y arrays de forma nativa.

Crear documentos en MongoDB

MongoDB permite insertar múltiples documentos utilizando `insertMany()`.

```
db.hotels.insertMany([
  {
    id: 1,
    name: "Grand Hotel",
    city: "Springfield",
    country: "USA",
    rating: 4.5,
    available_rooms: 20,
    price_per_night: 150.00,
    amenities: ["Free WiFi", "Pool", "Gym"],
    created_at: new Date(),
    updated_at: new Date()
  },
  {
    id: 2,
    name: "Cozy Inn",
    city: "Oakville",
    country: "USA",
    rating: 3.8,
    available_rooms: 15,
    price_per_night: 90.00,
    amenities: ["Free Breakfast", "Parking"],
    created_at: new Date(),
    updated_at: new Date()
  },
  {
    id: 3,
    name: "Luxury Suites",
    city: "Los Angeles",
    country: "USA",
    rating: 4.8,
    available_rooms: 10,
    price_per_night: 300.00,
    amenities: ["Spa", "Concierge", "Fine Dining"],
    created_at: new Date(),
    updated_at: new Date()
  }
])
```

Los documentos pueden contener distintos tipos de datos, incluyendo strings, números, fechas y arrays.



Operaciones CRUD y consultas

Crear

```
db.hotels.insertMany([...])
```

Leer

```
db.hotels.find().sort({ rating: 1 }).limit(2)
```

Actualizar

```
db.hotels.updateOne(  
  { name: "Grand Hotel" },  
  { $set: { address: "999 St" } }  
)
```

Eliminar

```
db.hotels.deleteOne({ name: "Grand Hotel" })  
db.hotels.deleteMany({ rating: { $lt: 3 } })
```

La combinación de `skip()` y `limit()` permite implementar paginación simple.

Conectando el Servicio a MongoDB

Inicializando la Conexión

```
package main

import (
    "context"
    "log"
    "time"

    "go.mongodb.org/mongo-driver/mongo"
    "go.mongodb.org/mongo-driver/mongo/options"
)

func connectDB() *mongo.Client {
    client, err :=
        mongo.NewClient(options.Client().ApplyURI("mongodb://root:root@localhost:27017"))
    if err != nil {
        log.Fatal(err)
    }

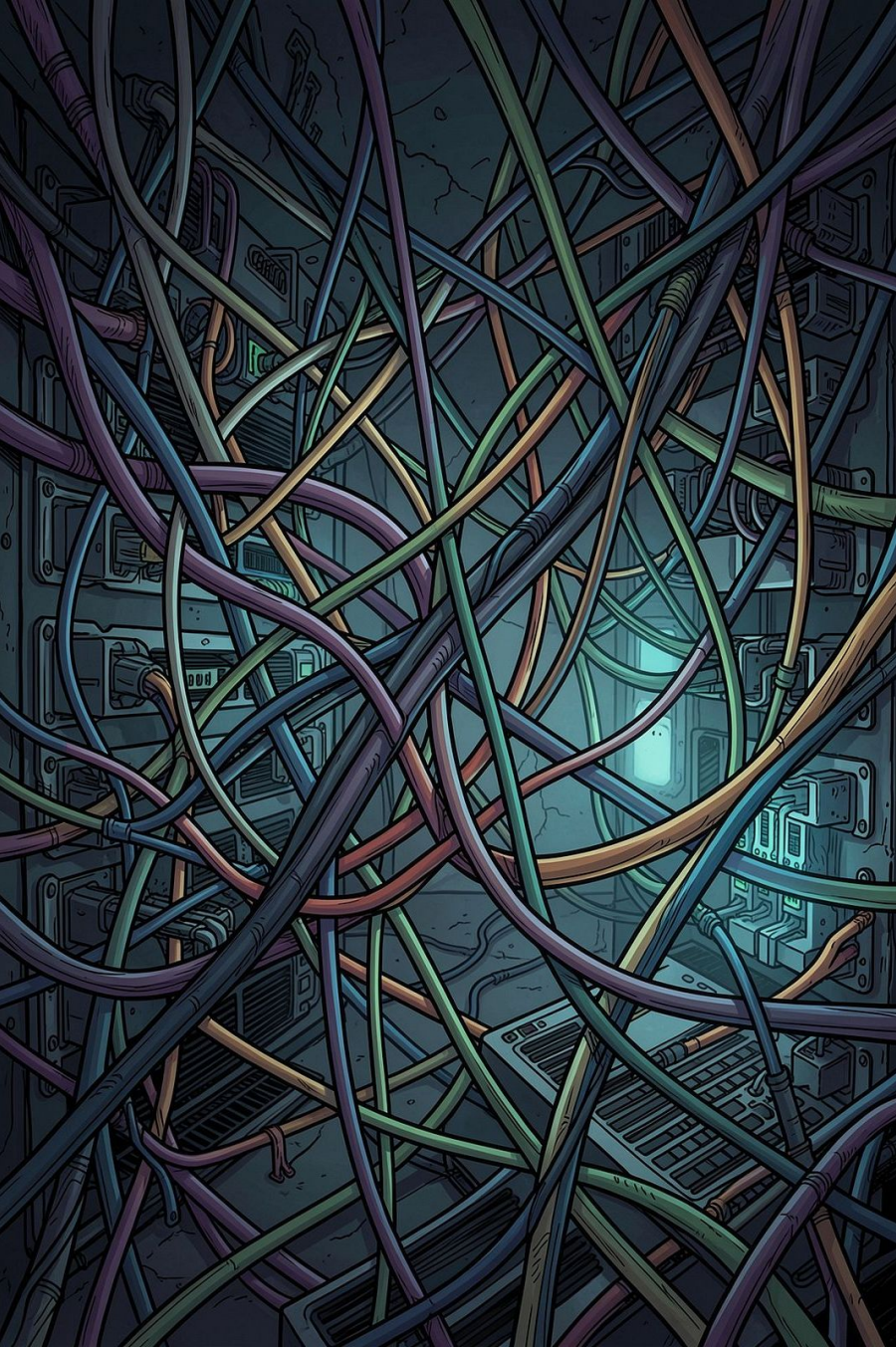
    ctx, cancel := context.WithTimeout(context.Background(),
        10*time.Second)
    defer cancel()

    err = client.Connect(ctx)
    if err != nil {
        log.Fatal(err)
    }

    log.Println("Conectado a MongoDB!")
    return client
}
```



Este fragmento esencial en Go establece la conexión con nuestra instancia de MongoDB, utilizando las credenciales `root/root` y el puerto `27017` configurados con Docker. Es el primer paso para interactuar con la base de datos.



EL PROBLEMA

¿Qué hacemos durante los tests?

Si el código accede directamente a MongoDB, los tests dependen de infraestructura externa: instancia activa, conexión disponible y datos precargados.

✗ Sin desacople

Tests lentos, frágiles y dependientes de MongoDB

✓ Con interfaces

Reemplazamos MongoDB por un Mock en memoria

Implementación real vs. implementación Mock

Podemos pensar dos formas diferentes de obtener un hotel.

Implementación real

```
mongo.Connect(ctx, clientOptions)
```

```
client.  
  Database("hotel-platform").  
  Collection("hotels").  
  Find(ctx, bson.M{"id": 1})
```

Se utiliza durante la ejecución real de la aplicación.

Implementación Mock

```
docs := make(map[string]HotelDAO)
```

```
hotel := docs[id]
```

Los datos viven en memoria y podemos controlarlos completamente durante los tests.

El objetivo es que la lógica de negocio no necesite saber cuál de las dos implementaciones está utilizando.

¿Cómo desacoplamos las implementaciones?

Queremos que nuestro Service pueda trabajar tanto con:

MongoDB

Mock

sin modificar su lógica.

Para conseguirlo utilizaremos una interfaz.

La interfaz define qué operaciones necesitamos, pero no define cómo se realizan.

Esto permite separar:

1

2

Contrato

Implementación

Tres pasos para desacoplar con interfaces



La interfaz define **qué** operaciones necesitamos, no **cómo** se realizan. Ambas implementaciones cumplen el mismo contrato `HotelsRepo`, permitiendo intercambiarlas sin modificar la lógica de negocio.

Paso 1: Definir la interfaz

```
type HotelsRepo interface {  
    GetHotelByID(id int64) (HotelDAO, error)  
}
```

La interfaz establece un contrato:

Cualquier implementación que pueda obtener un hotel por ID y respete esta firma puede funcionar como HotelsRepo.

La interfaz no debería conocer MongoDB, mocks ni detalles de infraestructura.

📌 **Concepto clave:** La interfaz debe ser genérica y representar las necesidades de nuestra aplicación.

Paso 2: Implementación MongoDB

Creamos una implementación real del repositorio:

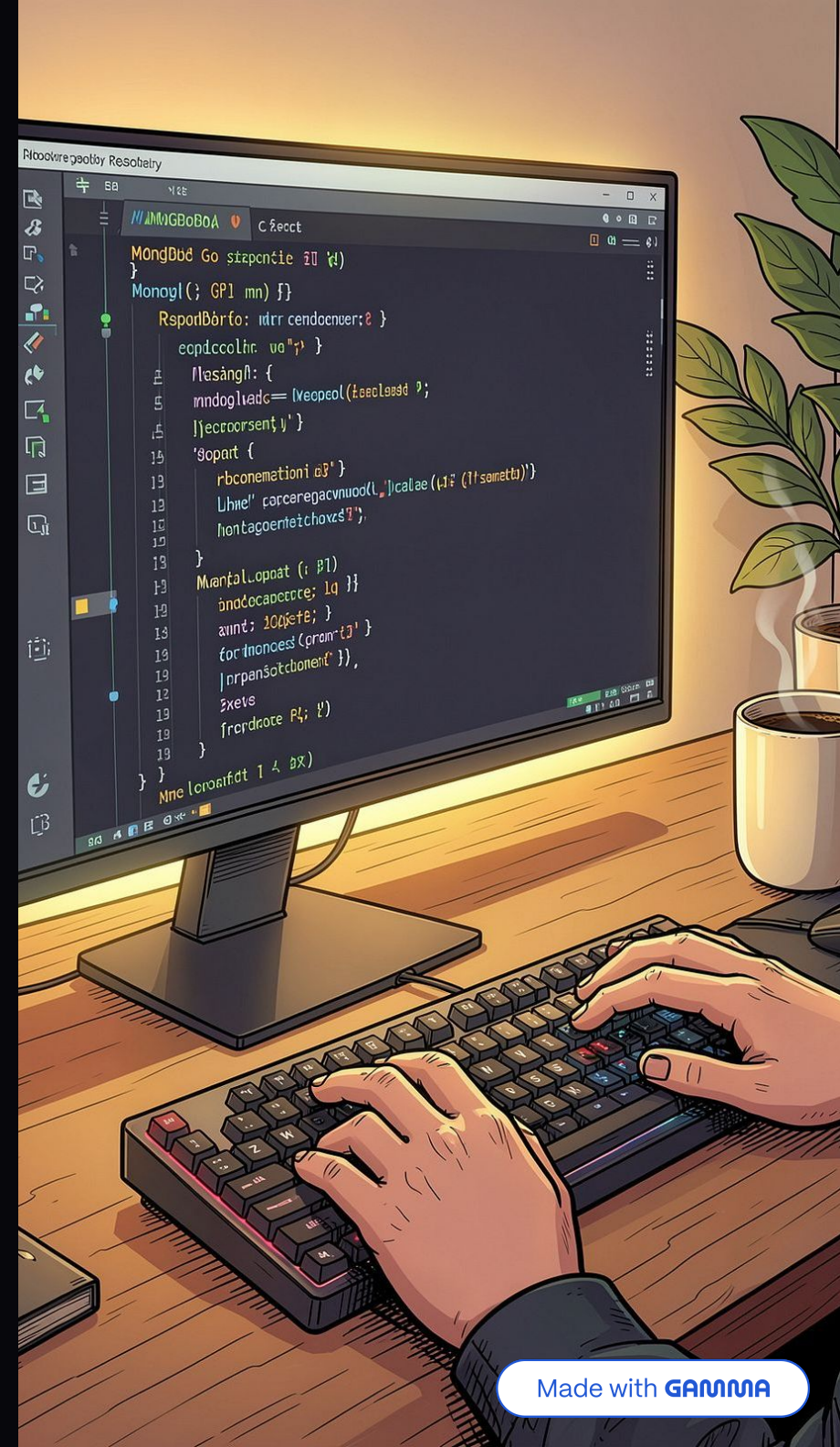
```
type HotelsMongoDB struct {
    client *mongo.Client
}
```

Luego implementamos el contrato:

```
func (repo HotelsMongoDB) GetHotelByID(id int64) (HotelDAO, error)
{
    // Consultar MongoDB
}
```

La implementación MongoDB contiene los detalles necesarios para acceder a la base de datos.

El resto de la aplicación no debería necesitar conocer esos detalles.



Paso 3: Implementación Mock

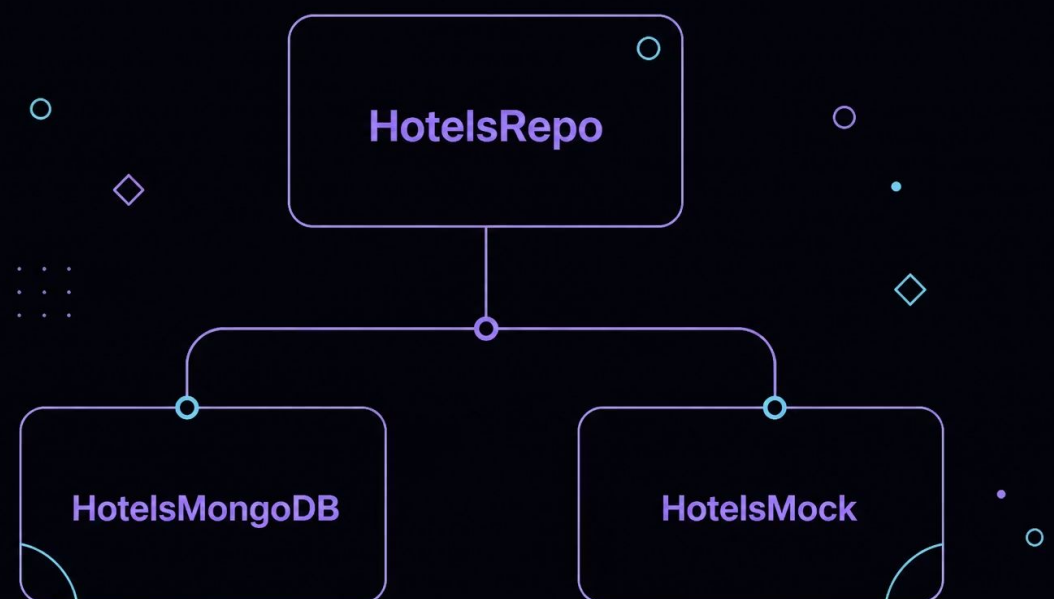
Creamos una segunda implementación:

```
type HotelsMock struct {  
    docs map[string]HotelDAO  
}
```

Y también implementamos:

```
func (repo HotelsMock) GetHotelByID(id int64) (HotelDAO, error) {  
    // Obtener hotel desde memoria  
}
```

Ahora tenemos dos implementaciones diferentes del mismo contrato:



MongoDB se utiliza para la ejecución real y Mock para los tests.

Paquete repositories: Implementaciones Clave

El paquete `repositories` encapsula la lógica de acceso a datos, ofreciendo una abstracción limpia y desacoplada de la base de datos subyacente.



Interfaz del Repositorio

Define los contratos para las operaciones de persistencia (e.g., `Save(item)`, `FindByID(id)`), garantizando la flexibilidad para cambiar la tecnología de la base de datos.



Implementación MongoDB

Contiene la lógica específica para interactuar con MongoDB, utilizando el driver oficial de Go. Traduce los objetos de dominio a documentos MongoDB y viceversa.



Mapeo de Datos

Define las estructuras Go que representan los documentos en MongoDB, manejando la serialización y deserialización para asegurar la compatibilidad entre el dominio y la persistencia.

Separación de Responsabilidades

La implementación real mantiene el cliente de MongoDB:

```
type HotelsMongoDB struct {  
    client *mongo.Client  
}
```

Y realiza consultas sobre:

```
repo.client.  
    Database("hotel-platform").  
    Collection("hotels")
```

La implementación Mock mantiene los datos en memoria:

```
type HotelsMock struct {  
    docs map[string]HotelDAO  
}
```

Conceptualmente:

```
func (repo HotelsMock) GetHotelByID(id int64) (HotelDAO, error) {  
    // buscar el hotel dentro de repo.docs  
}
```

Ambas implementaciones cumplen la misma interfaz, pero resuelven el acceso a datos de formas completamente diferentes.

Repository Pattern: Concepto Central

Lo que estamos implementando es una separación mediante repositorios.

Service

Contiene la lógica de negocio.

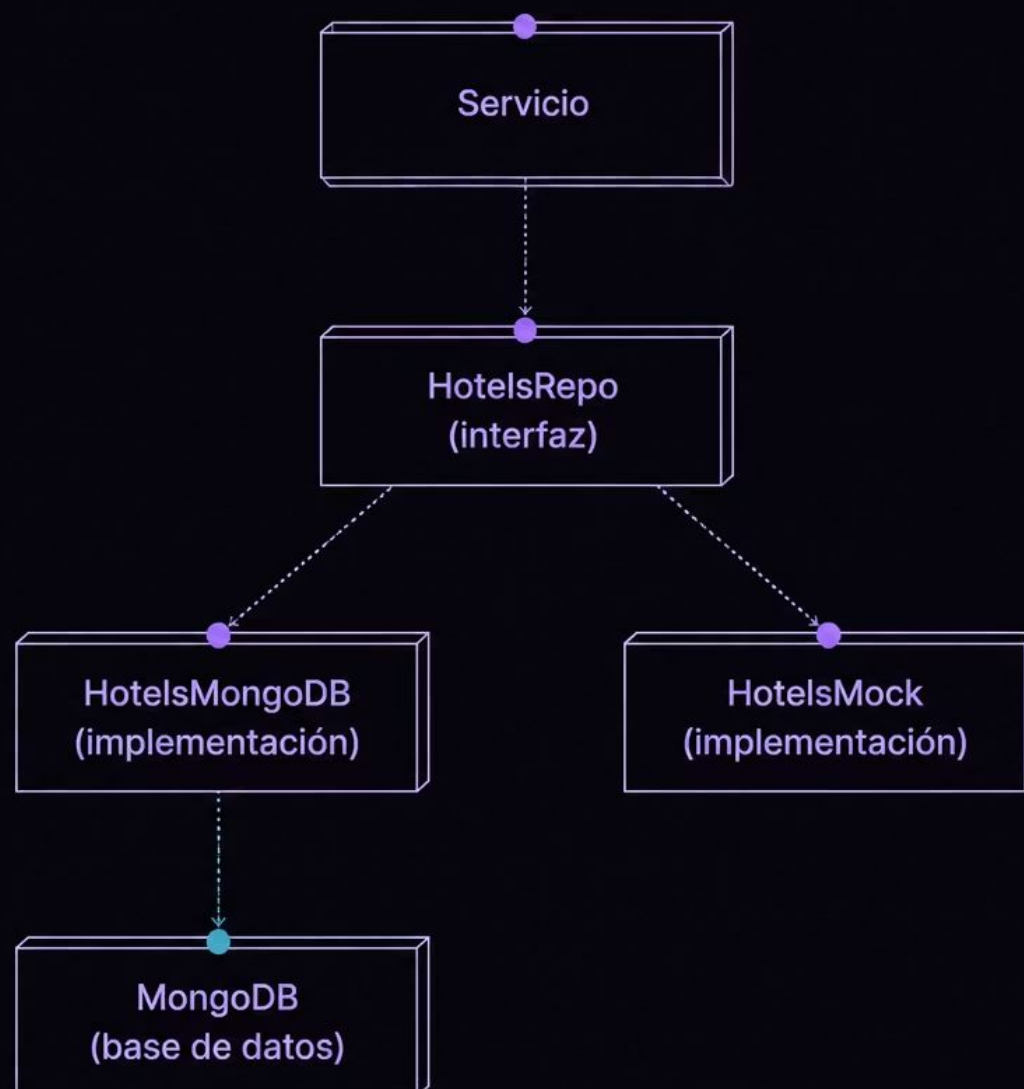
Repository

Abstrae el acceso a los datos.

MongoDB

Es un detalle concreto de infraestructura.

La dependencia debería verse conceptualmente así:

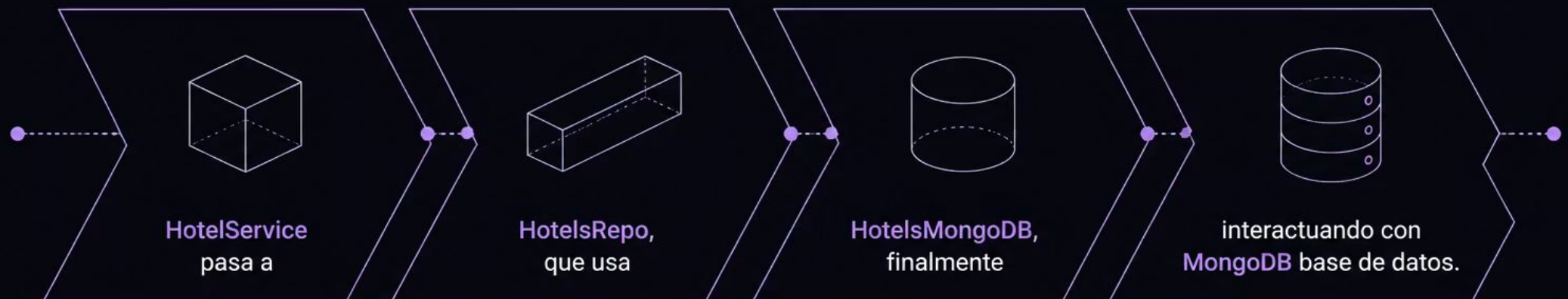


📌 El Service depende de una abstracción, no directamente de MongoDB.

Esto permite reemplazar la implementación sin modificar la lógica de negocio.

Service: Ejecución real con MongoDB

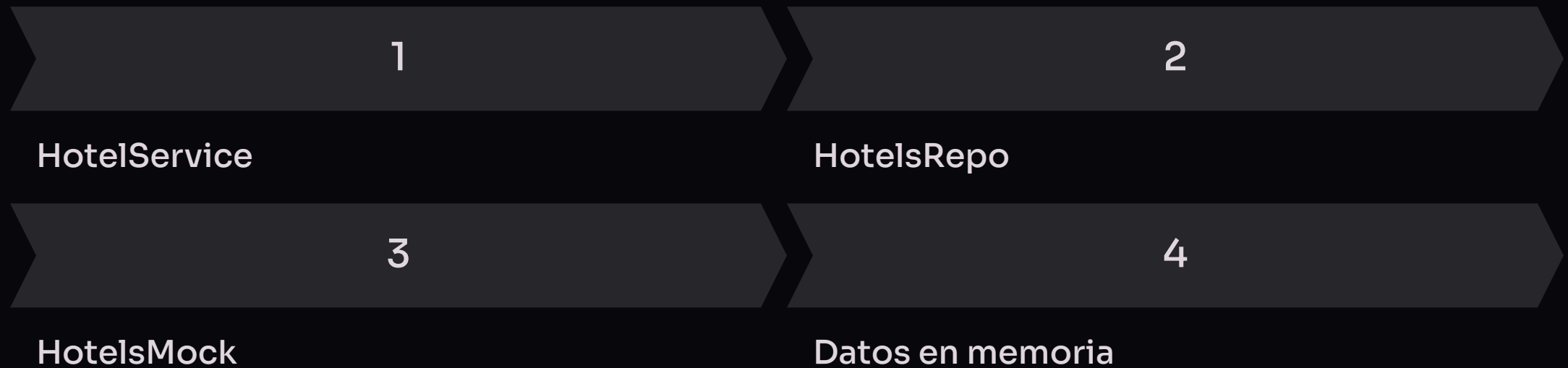
Durante la ejecución normal de la aplicación:



El Service trabaja con el repositorio y la implementación utilizada en producción es la correspondiente a MongoDB.

El Service no conoce los detalles de cómo se accede a MongoDB. Solo sabe que tiene un HotelsRepo que puede usar.

Service: Ejecución de tests con Mock



De esta forma podemos probar la lógica del Service sin depender de MongoDB.

El mismo código del Service funciona con ambas implementaciones. Solo cambia el repositorio que le inyectamos.

Producción vs. Test: Un Enfoque Estratégico

El Repository Pattern es crucial para desacoplar nuestra aplicación de la infraestructura de datos, permitiendo manejar de forma óptima los entornos de producción y prueba.

Producción: Realidad y Resiliencia

Conexión directa con **MongoDB real** y datos en vivo.

Prioridad: **Disponibilidad, integridad de datos y performance.**

Manejo de transacciones complejas y alta concurrencia.

Test: Agilidad y Fiabilidad

Uso de **mocks** o bases de datos en memoria para el repositorio.

Prioridad: **Aislamiento, velocidad y reproducibilidad** de las pruebas.

Facilita pruebas unitarias e integración sin dependencias externas.

Esta separación nos permite desarrollar y probar con confianza, sabiendo que los cambios en la lógica de negocio no se ven afectados por la base de datos subyacente.

Flujo completo de la arquitectura



Esta visualización ilustra cómo una solicitud fluye desde el cliente, pasando por nuestra aplicación Go, hasta interactuar con la base de datos MongoDB, destacando la abstracción que proporciona el Repository Pattern.

Ejercicio práctico: API de Items con MongoDB

Es hora de aplicar lo aprendido. Construiremos una API RESTful simple para gestionar "items", integrando Go, MongoDB y el Repository Pattern.

Modelado de Ítems

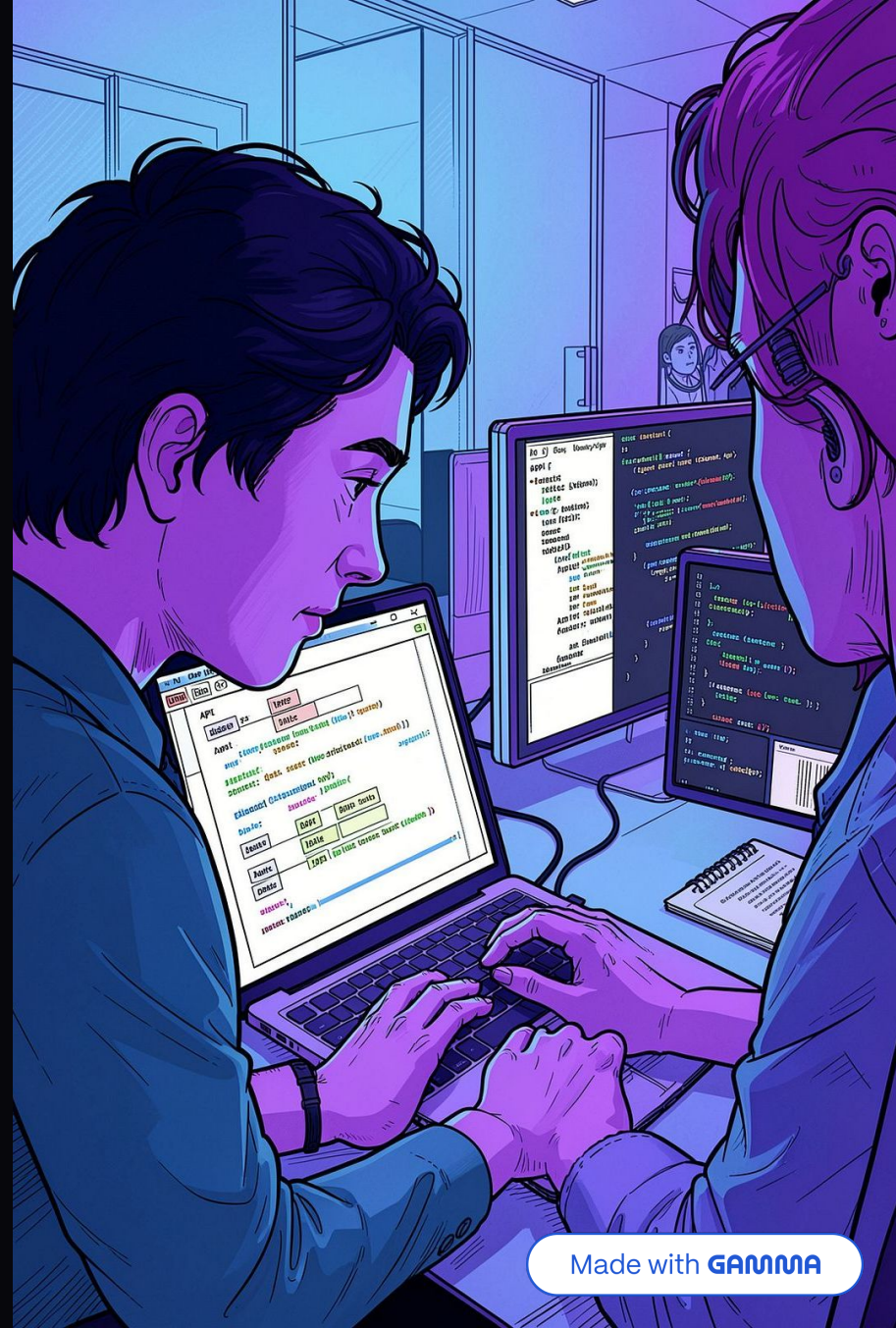
Definiremos la estructura de datos para nuestros ítems.

Repository para MongoDB

Implementaremos las operaciones CRUD sobre una colección de ítems.

Exposición REST

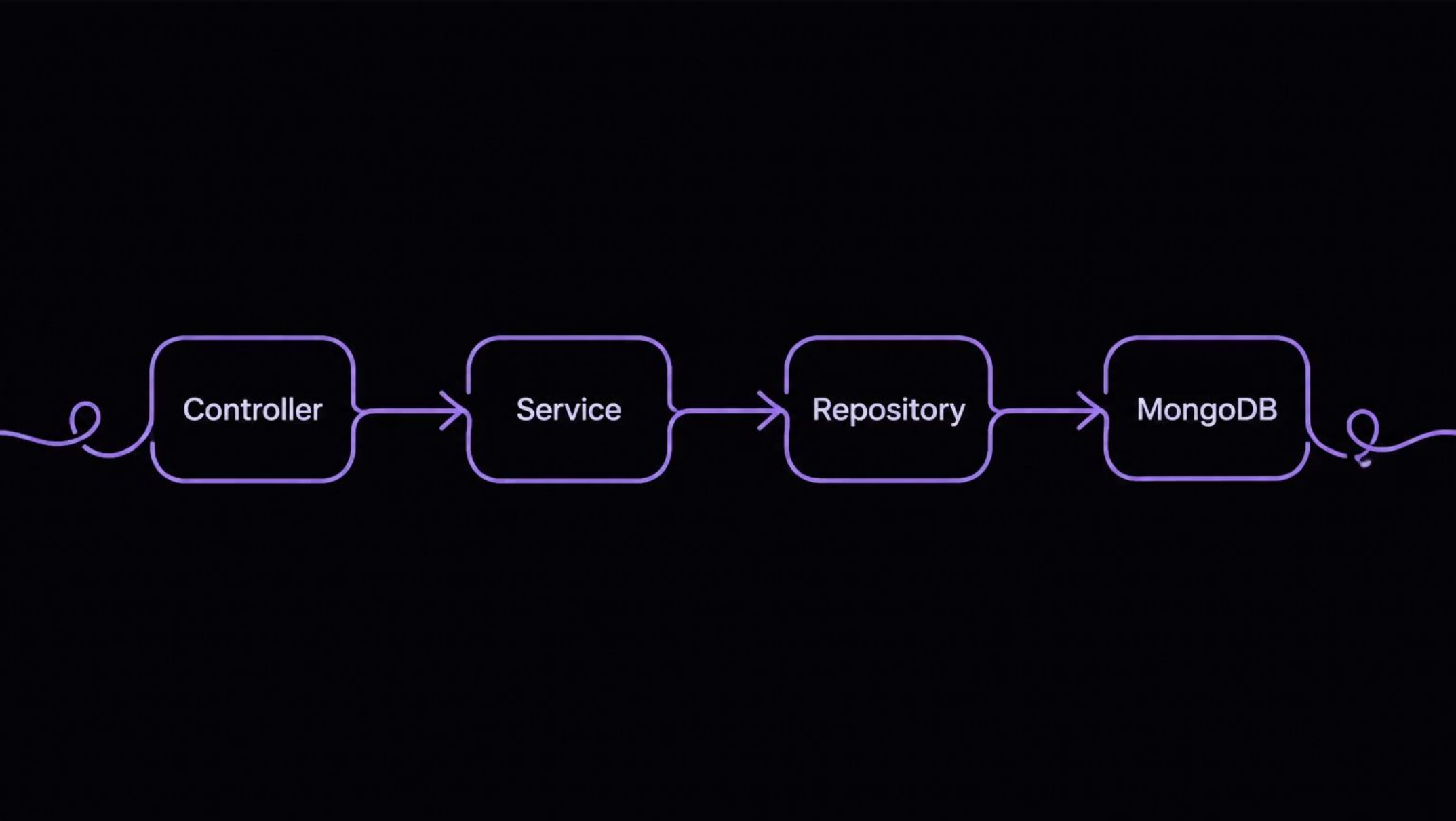
Crearemos los endpoints HTTP para interactuar con la API.



Qué debería respetar la solución

Repository Responsable del acceso a MongoDB. Ejemplos de operaciones: • GetItemByID • CreateItem • UpdateItem • DeleteItem	Service Responsable de la lógica de negocio y debe utilizar la abstracción del repositorio.	Controller Responsable de: • recibir la request HTTP; • obtener parámetros y body; • llamar al Service; • devolver la response HTTP.
--	--	---

Principio fundamental:



Cada capa debe tener una responsabilidad clara.

Principios Clave del Diseño de Repositorios

Para asegurar una arquitectura robusta y desacoplada, nuestra implementación del Repository Pattern debe adherirse a las siguientes directrices:

Independencia del ORM/Base de Datos

La capa de negocio no debe conocer los detalles de la tecnología de persistencia (ej. MongoDB, SQL). Solo se interactúa con la interfaz del repositorio.

Testabilidad Aumentada

Facilitar el testing unitario y de integración al permitir que la implementación real de la base de datos sea sustituida por mocks o stubs.

Separación de Responsabilidades

Cada repositorio debe ser responsable exclusivamente de la gestión de una entidad agregada, encapsulando las operaciones CRUD.

Flexibilidad y Mantenibilidad

Permitir cambios en la tecnología de almacenamiento con el menor impacto posible en el resto de la aplicación.

Organización del proyecto Go

```
hotels-api/  
├── dao/hotels/  
│   └── hotels_dao.go  
├── repositories/hotels/  
│   ├── hotels_mongo.go  
│   └── hotels_mock.go  
├── services/hotels/  
│   ├── hotels_service.go  
│   └── hotels_service_test.go  
├── go.mod  
└── main.go
```

Cada capa tiene una responsabilidad clara

→ **DAO**

Estructuras de datos

→ **Repository**

Acceso a datos (MongoDB o Mock)

→ **Service**

Lógica de negocio

→ **Controller**

HTTP request/response

¿Qué ganamos utilizando interfaces?



Desacoplamiento

La lógica de negocio deja de depender directamente de MongoDB.



Testabilidad

Podemos ejecutar tests utilizando un repositorio Mock sin levantar una base de datos.



Flexibilidad

Podemos agregar otra implementación en el futuro sin cambiar el contrato utilizado por el Service.

Por ejemplo:

HotelsRepo

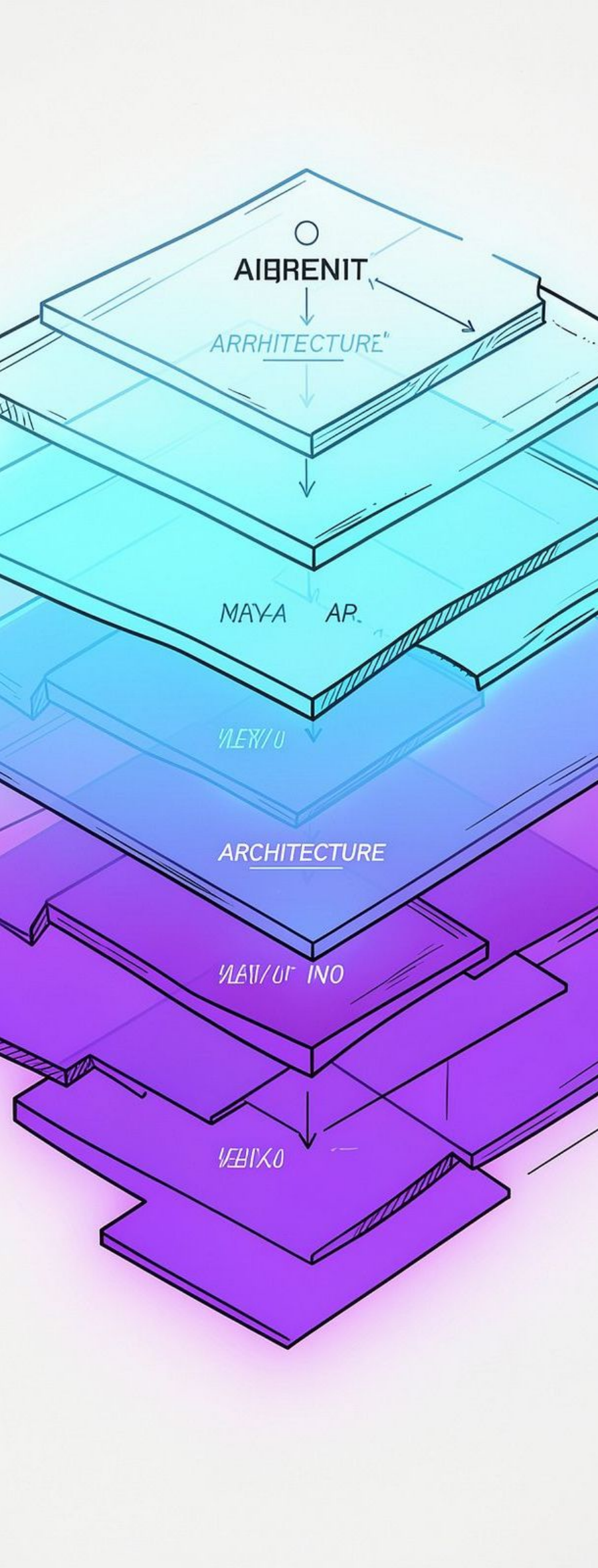
- HotelsMongoDB
- HotelsMock
- HotelsMySQL



Separación de responsabilidades

Cada componente tiene una función clara:

- DAO → representa los datos.
- Repository → acceso a datos.
- Service → lógica de negocio.
- Controller → entrada/salida de la API.



RESUMEN

Conceptos clave de la clase



MongoDB + Docker

Instancia local para desarrollo y pruebas



Interfaces en Go

Contratos que abstraen el acceso a datos



Repository Pattern

Service depende de una abstracción, no de MongoDB



Mock para tests

Tests rápidos sin infraestructura externa



Idea central: La lógica de negocio no debe depender de MongoDB. Debe depender de una abstracción que permita reemplazar la implementación concreta cuando sea necesario.